

MNIST Dataset

Unsupervised Learning, K-Means,
Dimensionality Reduction

Last week

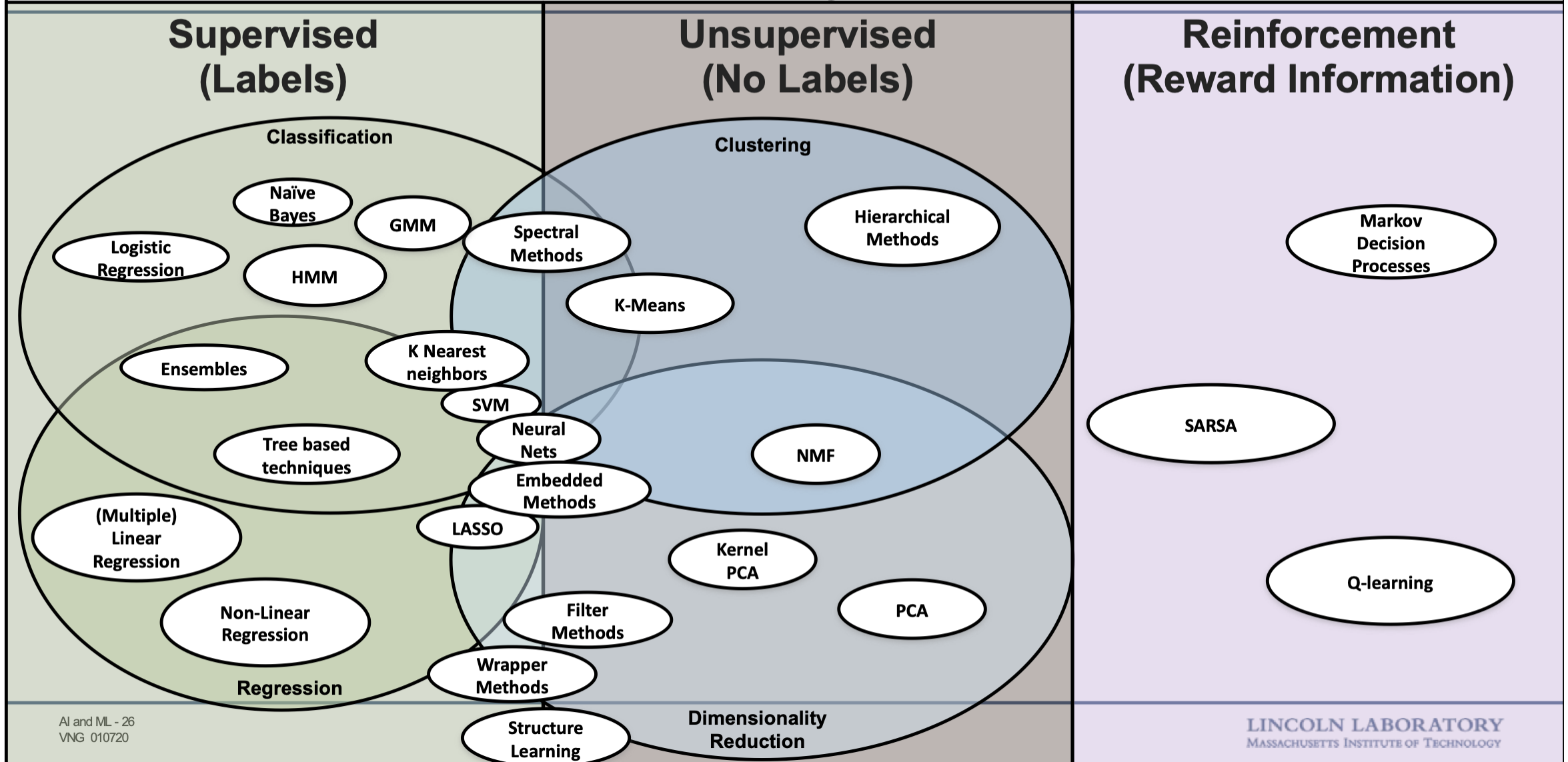
- Decision Trees
- Metrics: Gini Impurity, Entropy and Information Gain
- Ensemble Methods:
 - Random Forests (a bagging technique)
 - Boosting (gradients):
 - AdaBoost
 - Gradient Boosting
 - XGBoost
 - Voting and averaging

This week

- Unsupervised Learning (vs Supervised Learning)
- Clustering algorithms: K-Means
 - Metrics for evaluation
- Dimensionality Reduction (PCA)



Machine Learning Techniques



Supervised Learning

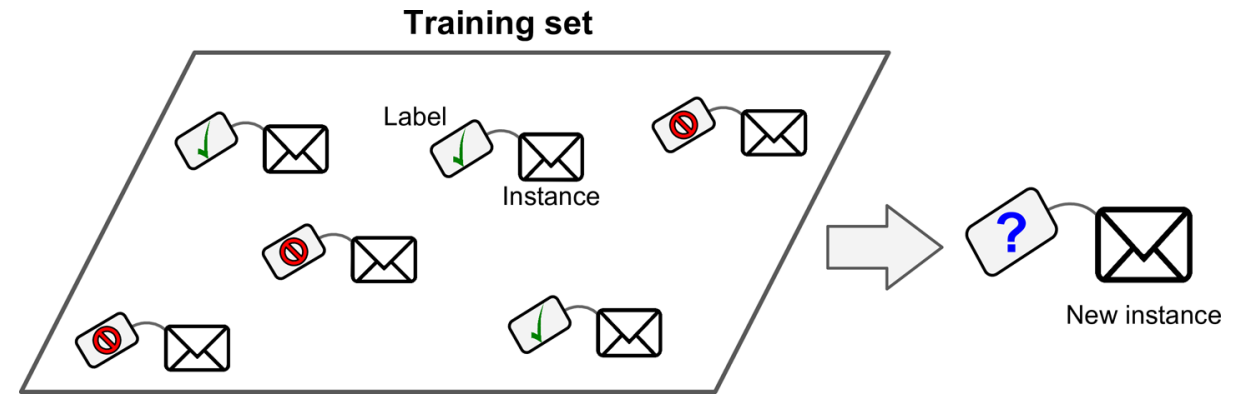
- My dataset is labelled and can be decomposed into a matrix of input features and an arrays of labels
- A supervised learning algorithm learns to predict the labels from the features:

X = feature matrix						y = label array/matrix	
Feat 1	Feat 2	Feat 3	Feat 4		Feat 5	Label	
1	4.5	good	Hello now!		yes	correct	
2	5.6	average	goodbye		no	incorrect	
...	...	...			...	...	
6	8.8765	bad	Ahaha, yes		no	incorrect	

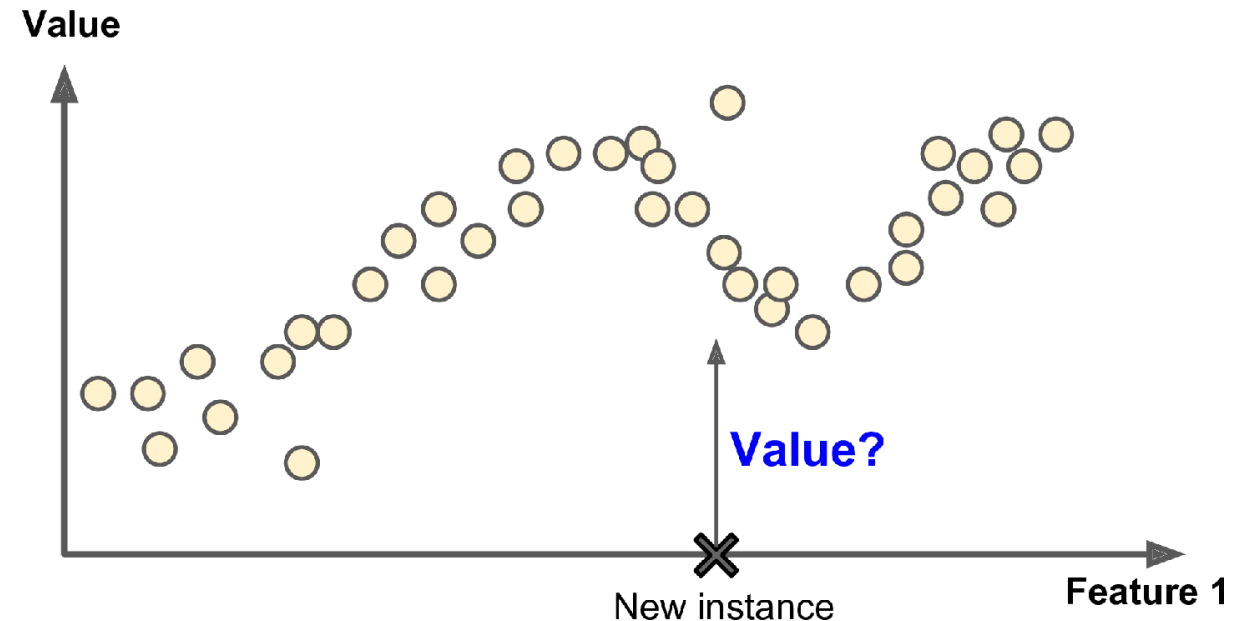
- The algorithm will learn the $X \Rightarrow y$ mapping i.e. $y = f(X)$
- If it has learned well its **y predictions** will be a match (or close) to the actual **y values**

Supervised Learning

- Classification: to which class does an instance belong to?



- Regression: given a set of pairs (X_i, Y_i) how can I fit them to a function $y = f(X)$?



Supervised Learning: examples

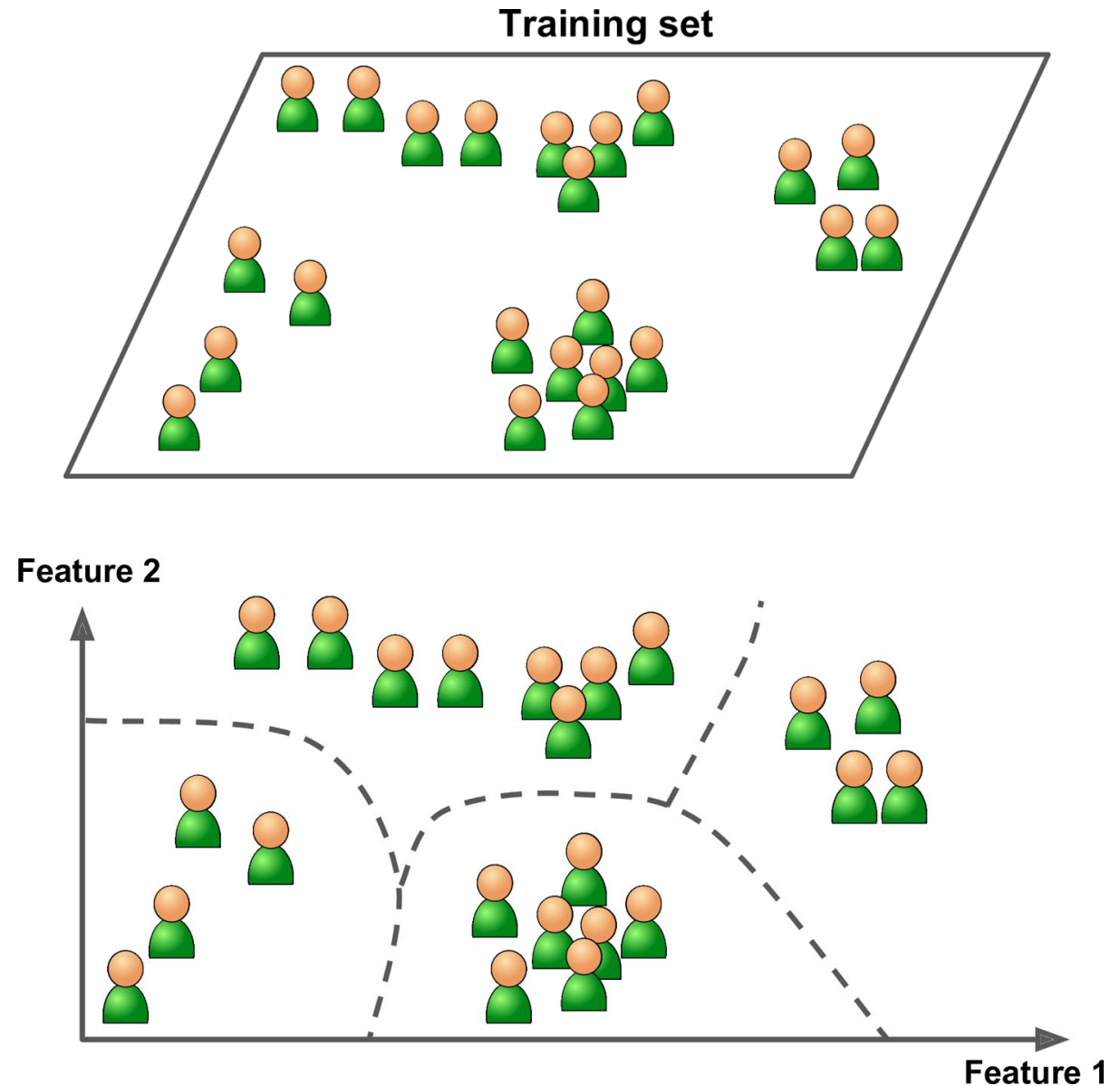
- k-Nearest Neighbors
- Linear Regression
- Logistic Regression
- Naïve Bayes Classification
- Support Vector Machines (SVMs)
- Gaussian Processes
- Decision Trees and Random Forests
- Ensemble Methods
- Neural networks

Unsupervised Learning

Finding previously unknown patterns in data set without pre-existing labels.

Examples:

- Clustering (hierarchical clustering, K-means, DBSCAN, gaussian mixtures)
- Anomaly Detection
- Dimensionality Reduction (PCA, Locally Linear Embedding)
- Neural Networks (Autoencoders, Deep belief nets, Generative adversarial networks)



Source: Aurélien Géron, *Hands-On Machine Learning*

Unsupervised Learning

- “If intelligence was a cake, unsupervised learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake.”
(Yann Le Cun)
- The vast majority of the available data is un-labelled: we have the input features **X**, but we do not have the labels **y** => enter **unsupervised learning**

Unsupervised Learning methods

- K-means (clustering)
- DBSCAN (clustering, anomaly detection)
- Mixture Models (clustering, density estimation, anomaly detection)
 - Gaussian Mixture Models
- Neural Networks
 - Autoencoders (dimensionality reduction)
 - Generative Adversarial Networks (generative model for unsupervised learning; semi/self-supervised; reinforcement learning)

Clustering

- Exclusive clustering => **k-means**
- Agglomerative => **Hierarchical clustering**
- Overlapping => **Fuzzy C-Means**



sample



Cluster/group

Clustering algorithms

- K-means (clustering)
 - very scalable
 - need to predefine number of clusters
 - convex-shaped clusters
- DBSCAN, HDBSCAN
 - No assumption on cluster shape
- Hierarchical clustering
 - Agglomerative clustering
- Gaussian Mixture Methods
- ... and many more ...

Introduction to K-Means Clustering

Introduction to K-Means Clustering

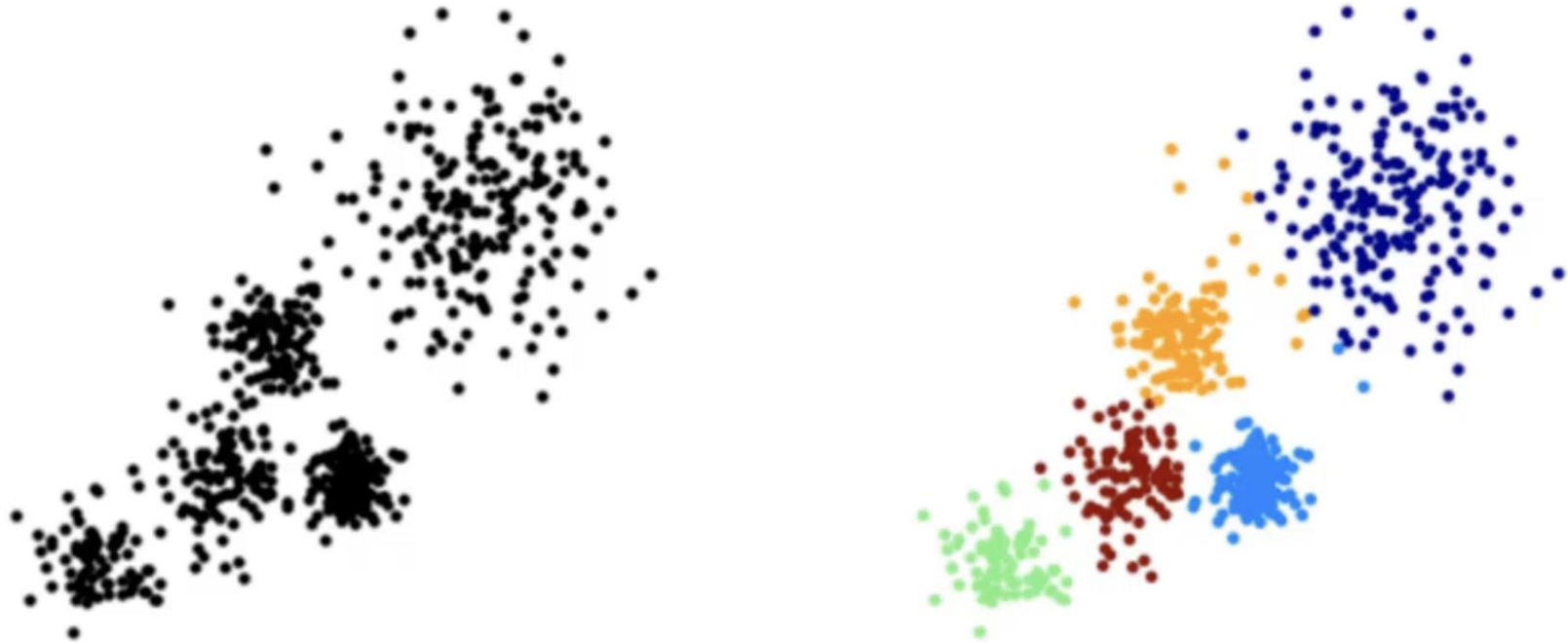
- K-Means Clustering is **not to be confused** with K-NN (which is supervised learning technique)
- K-Means Clustering is an **unsupervised** learning algorithm that will attempt to group similar clusters together in your data.
- So, what does a typical clustering problem look like?
 - Cluster similar documents
 - Cluster customers based on features
 - Market segmentation
 - Identify similar physical groups

Introduction to K-Means Clustering

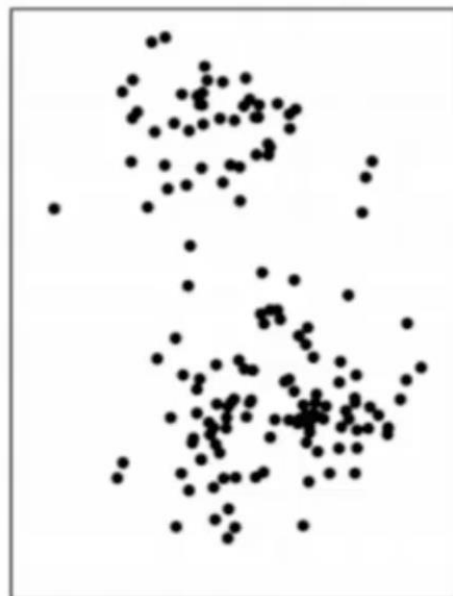
- The K-Means Algorithm
- Choose a number of Clusters 'K'
- Randomly assign each point to a cluster
- Until clusters stop changing, repeat the following:
 - For each cluster, compute the cluster centroid by taking the mean vector of points in the cluster
 - Assign each data point to the cluster for which the centroid is the closest.

K-Means Clustering

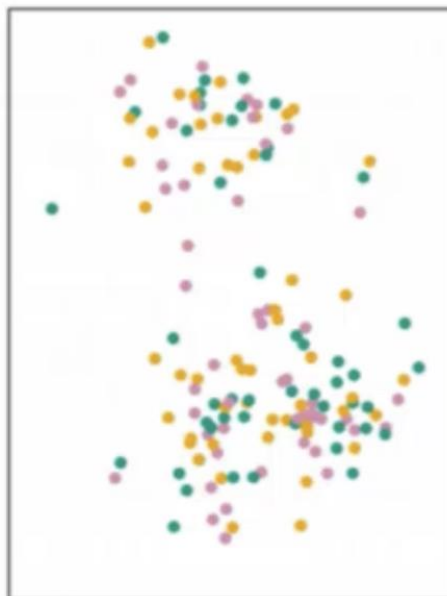
- The overall goal is to divide data into distinct groups such that observations within each group are similar



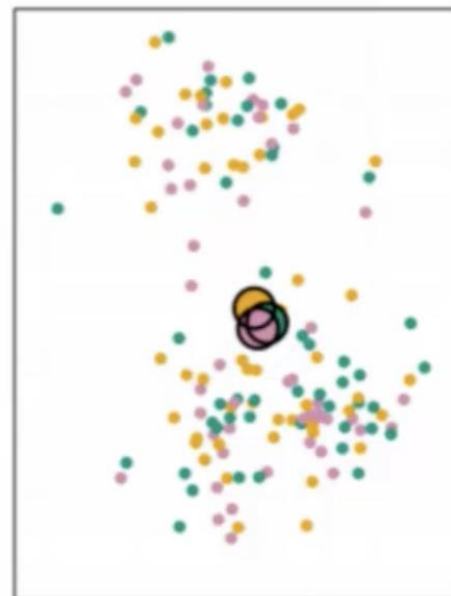
Data



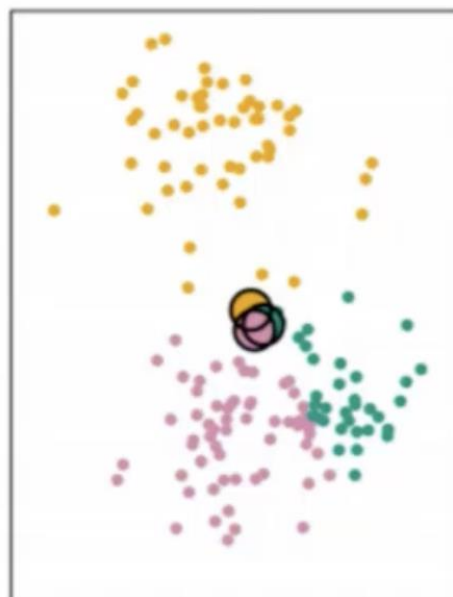
Step 1



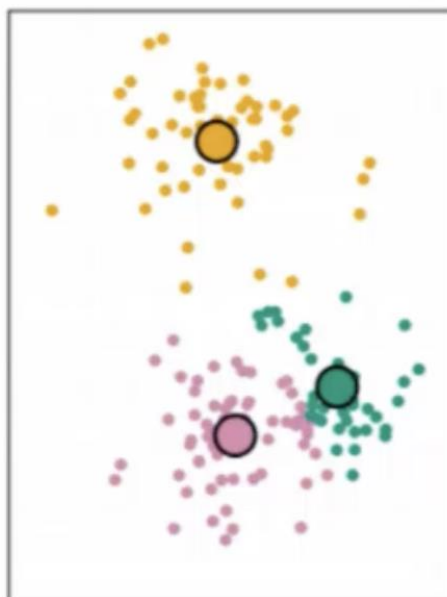
Iteration 1, Step 2a



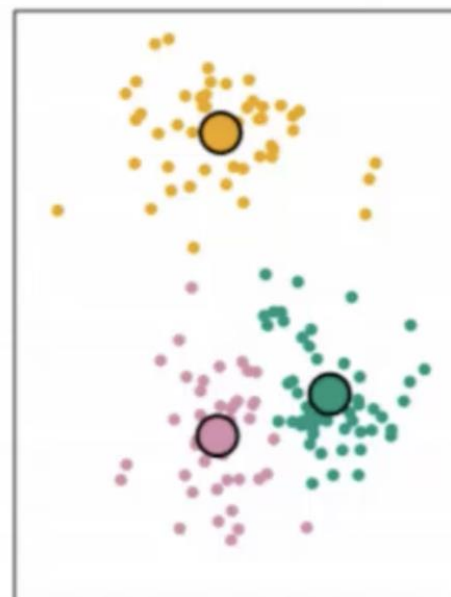
Iteration 1, Step 2b



Iteration 2, Step 2a



Final Results

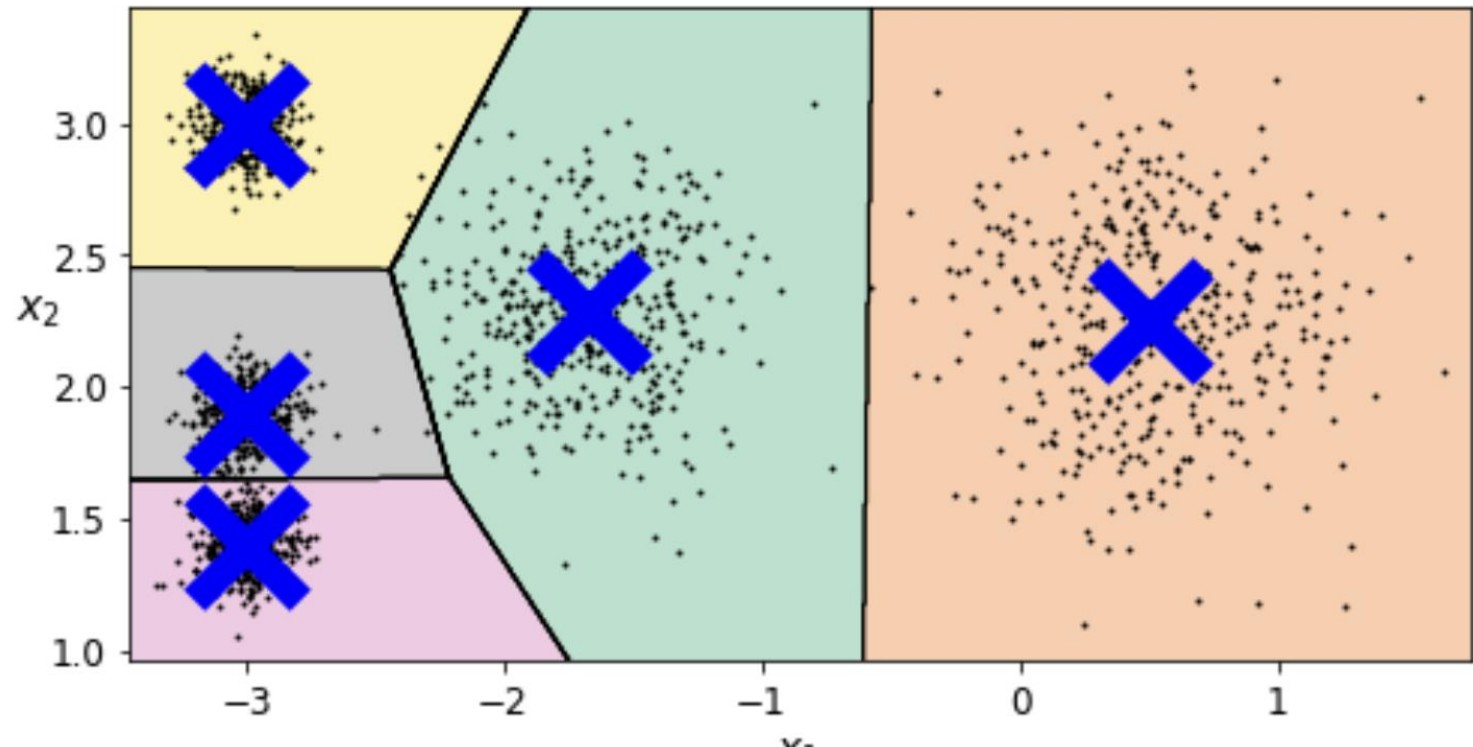


K-Means with SKlearn

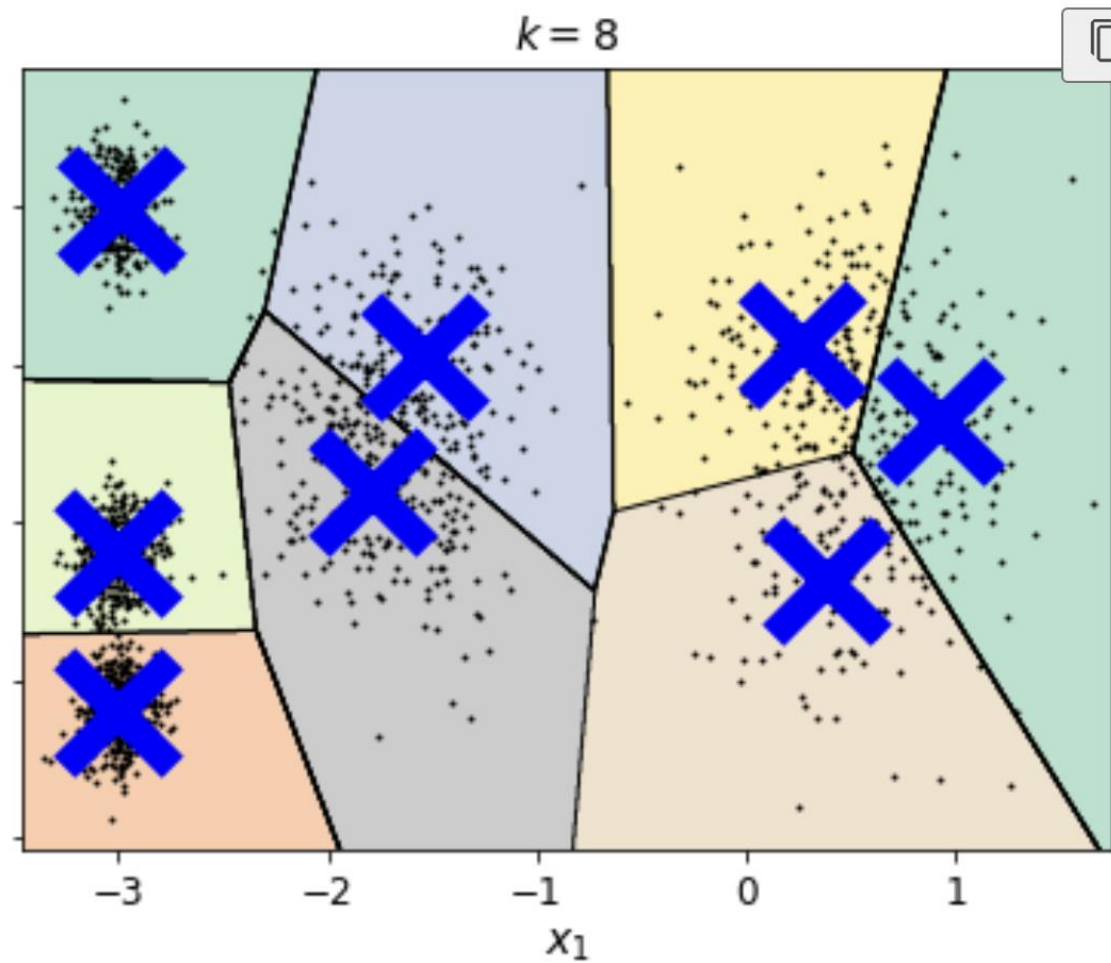
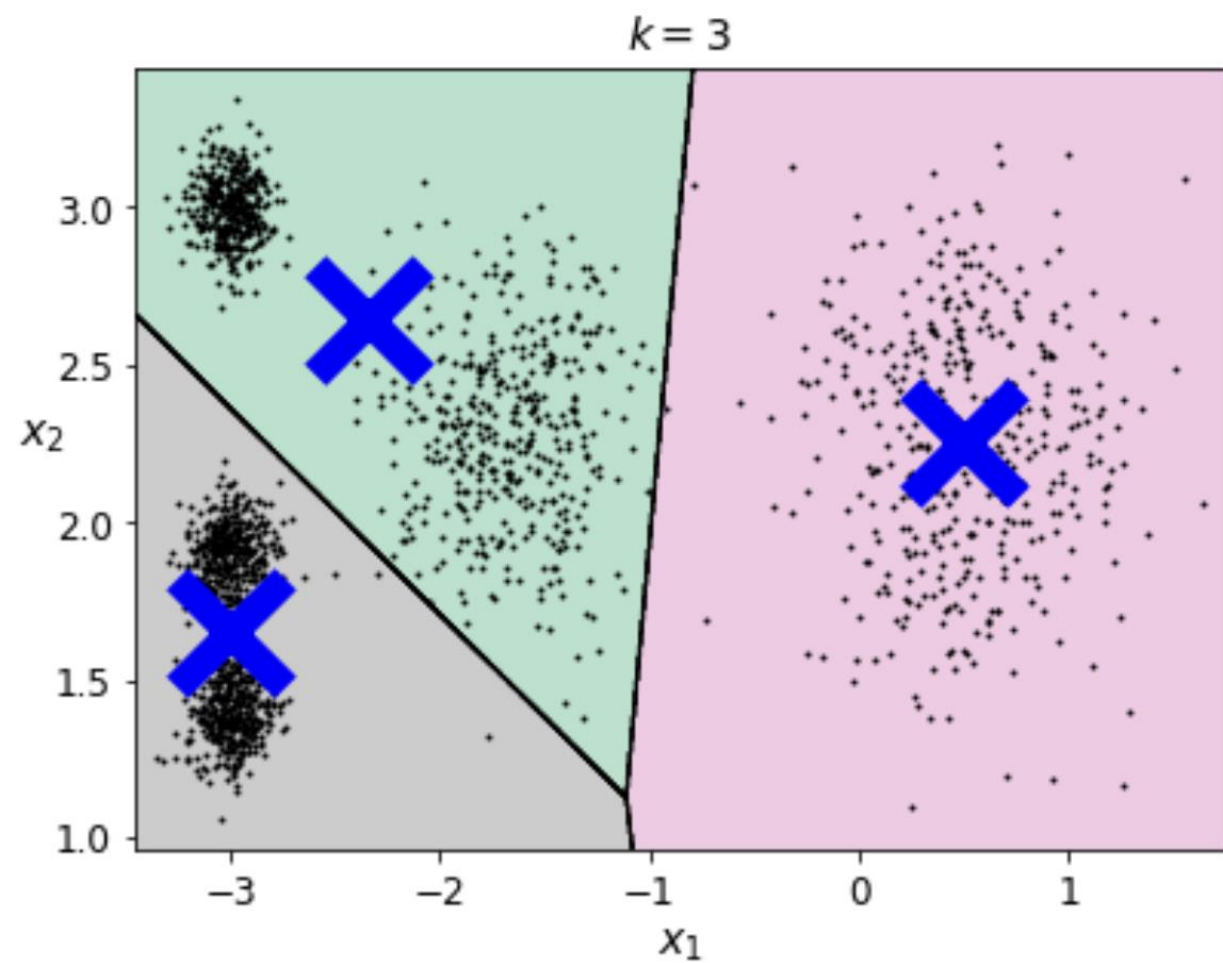
- from **sklearn.cluster** import **KMeans**
- k = 5
- kmeans = KMeans(
 n_clusters=k,
 n_init=10, # this is the default value. The algorithm will be run
10 times to improve convergence.
 random_state=42
)
- y_pred = kmeans.fit_predict(X)

K = 5

- `from clustering_utils import plot_decision_boundaries`
- `plt.figure(figsize=(8, 4))`
- `plot_decision_boundaries(kmeans, X)`
- `plt.show()`



$K = 3 / K = 8$



Choosing a K Value – Elbow method / Inertia

- There is no easy answer for choosing a “best” K value
- One way is the elbow method

First of all, compute the sum of squared error (SSE) for some values of k (for example 2, 4, 6, 8 etc).

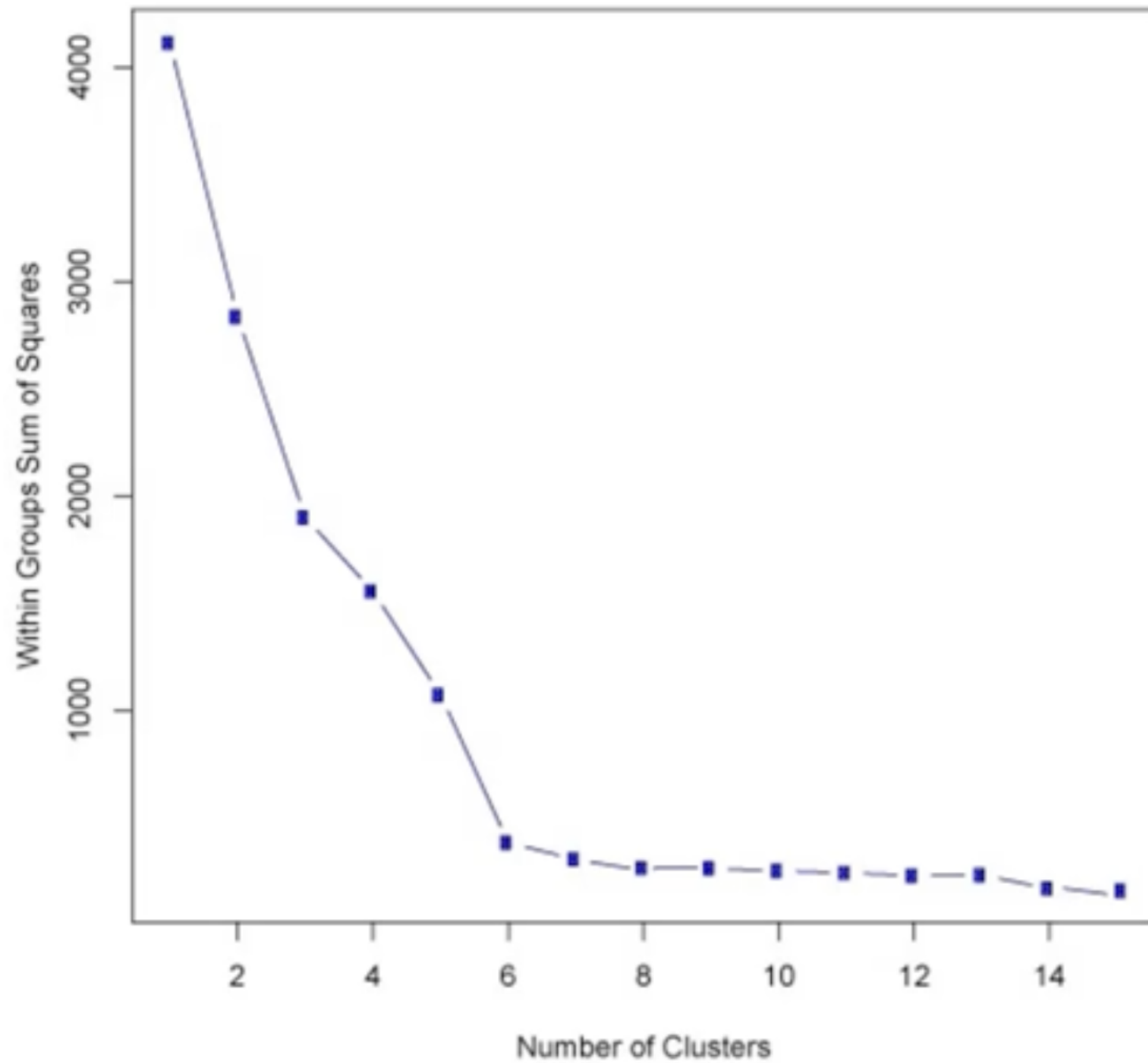
The SSE is defined as the sum of the squared distance between each member of the cluster and its centroid.

Inertia, also known as within-cluster sum-of-squares, measures the compactness of clusters. It calculates the total variance within the clusters. In simpler terms, it's the sum of the distances of each data point in a cluster to the centroid of that cluster, squared and summed up for all clusters

Choosing a K Value

- If you plot k against the SSE, you will see that the error decreases as k gets larger; this is because when the number of clusters increases, they should be smaller, so distortion is also similar
- The idea of the elbow method is to choose the k at which the SSE decreases abruptly.
- This produces an “elbow effect” in the graph as you see on the next slide:

Elbow



Inertia Scores

- `kmeans_per_k = [`
 `KMeans(n_clusters=k, random_state=42).fit(X)`
 `for k in range(1, 10)`
 `]`
- `inertias = [model.inertia_ for model in kmeans_per_k]`

```
kmeans_k3.inertia_, kmeans_k8.inertia_
```

```
(710.8168116555955, 120.56311113669005)
```

Unsupervised Learning methods

- Dimensionality Reduction
 - Matrix decomposition (PCA, LDA, LSA, ICA ...)
 - Manifold Learning
- Clustering
- Anomaly and Novelty Detection
- Covariance and Density Estimation
- Unsupervised Neural Networks
 - Restricted Boltzmann Machines
 - Autoencoders (dimensionality reduction, denoising)
 - Generative Adversarial Networks

Anomaly and Novelty Detection

- Local Outlier Factor (LOF)
- Isolation Forests
- One-class Support Vector Machines
- Elliptic Envelope
 - Gaussian distributed datasets

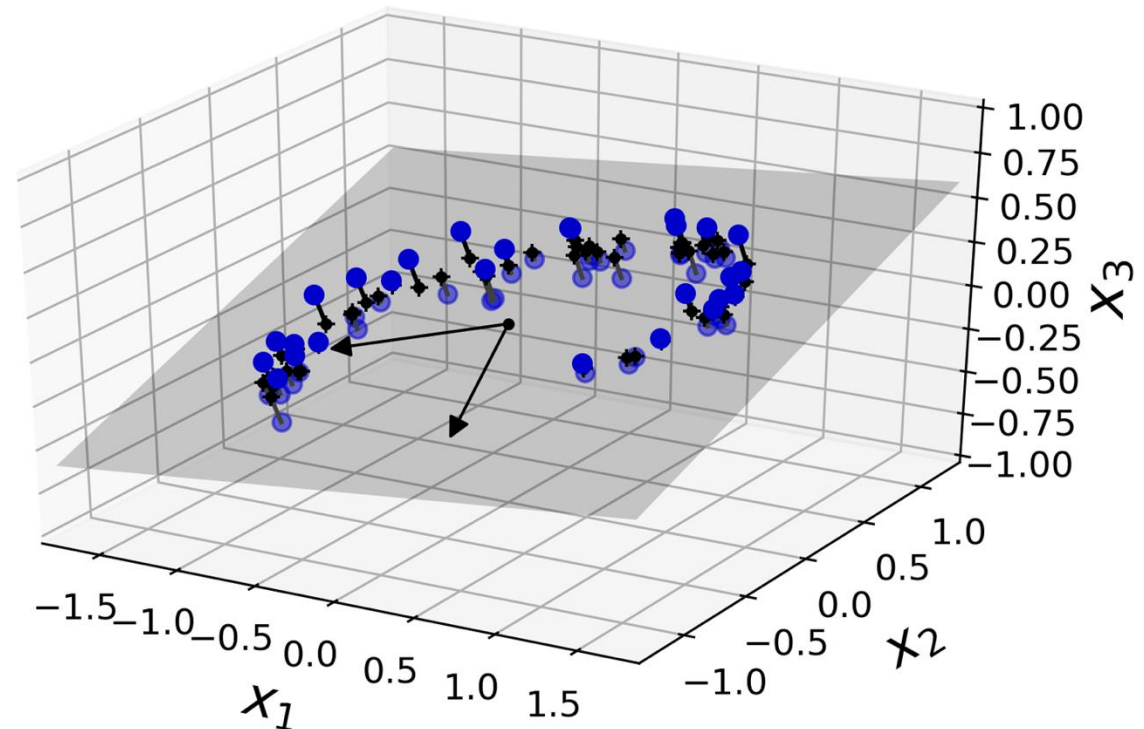
The Curse of Dimensionality

Difficulty of using brute force (a.k.a. grid search) to optimize a function with too many input variables.

- If we have more features than observations than **we run the risk of massively overfitting our model — this would generally result in terrible out of sample performance.**
- When we have too many features, observations become harder to cluster => **too many dimensions causes every observation in your dataset to appear equidistant from all the others.**

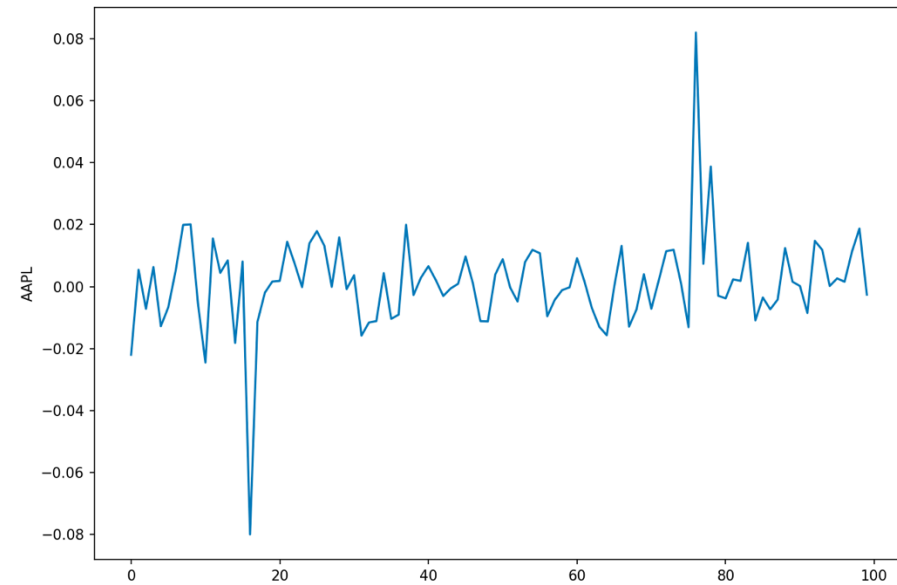
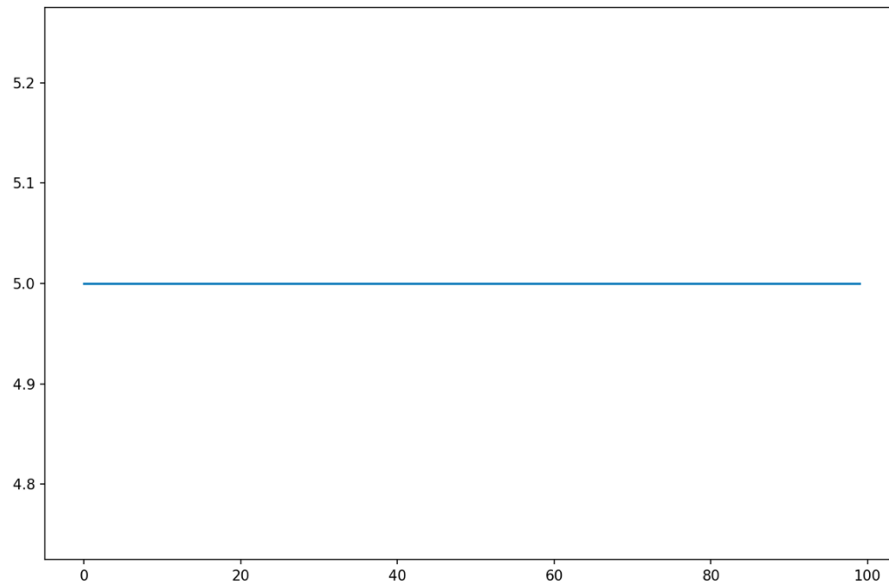
Projection

In most real-world problems, training instances are *not* spread out uniformly across all dimensions. Many features are almost constant, while others are highly correlated. As a result, all training instances lie within (or close to) a much lower-dimensional *subspace* of the high-dimensional space.



Source: Aurélien Géron, ***Hands-On Machine Learning***, 2nd ed., O'Reilly

Signal and Variance



No variance, no information!

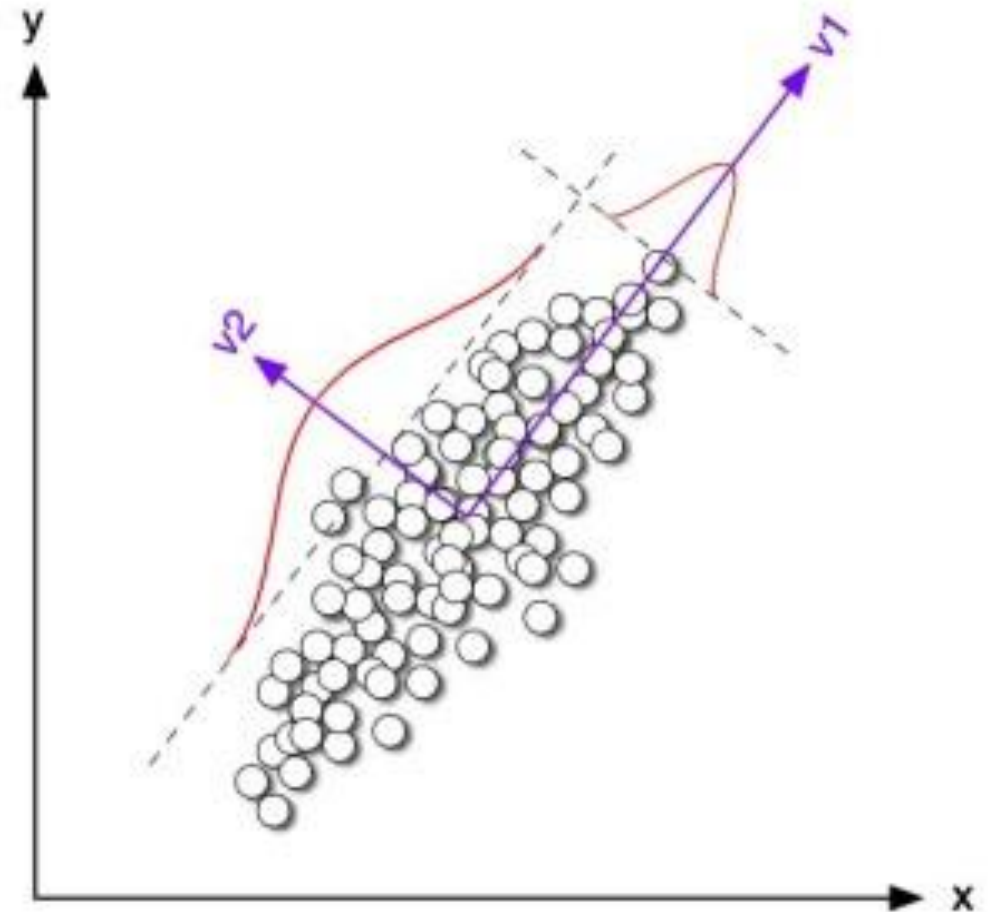
The ideal set of features

- **High Variance:** features with a lot of variance contain a lot of potential signal
- **Uncorrelated:** features that are highly correlated with each other are less useful and in certain cases downright harmful
- **Not That Many:** We want to have a low number of features relative to our number of target variable observations.

Introduction to Principal Component Analysis (PCA)

Principal Component Analysis

PCA creates a set of principal components that are rank ordered, uncorrelated, and low in number (we can throw away the lower ranked components as they contain little signal).



Principal Component Analysis

- Principal Component Analysis (PCA) was first introduced by **Karl Pearson**, a British mathematician, in 1901.
- Pearson developed this method to analyse and simplify complex data sets by reducing the dimensionality while preserving most of the variance in the data.
- It is an unsupervised statistical technique used to examine the interrelations among a set of variables to identify the underlying structure of those variables.
- It is sometimes known as general **factor analysis**.



Background to dimensionality

- Where regression determines a line of best fit to a data set, factor analysis determines several orthogonal lines of best fit to the data set.
- Orthogonal means “at right angles”.
 - Actually the lines are perpendicular to each other in ‘n-dimensional space’.
- ‘N-Dimensional Space’ is **the variable sample space**.
 - There are as **many dimensions as there are variables**
 - E.g. in data **set with 4 variables, the sample space is 4 dimensional**.

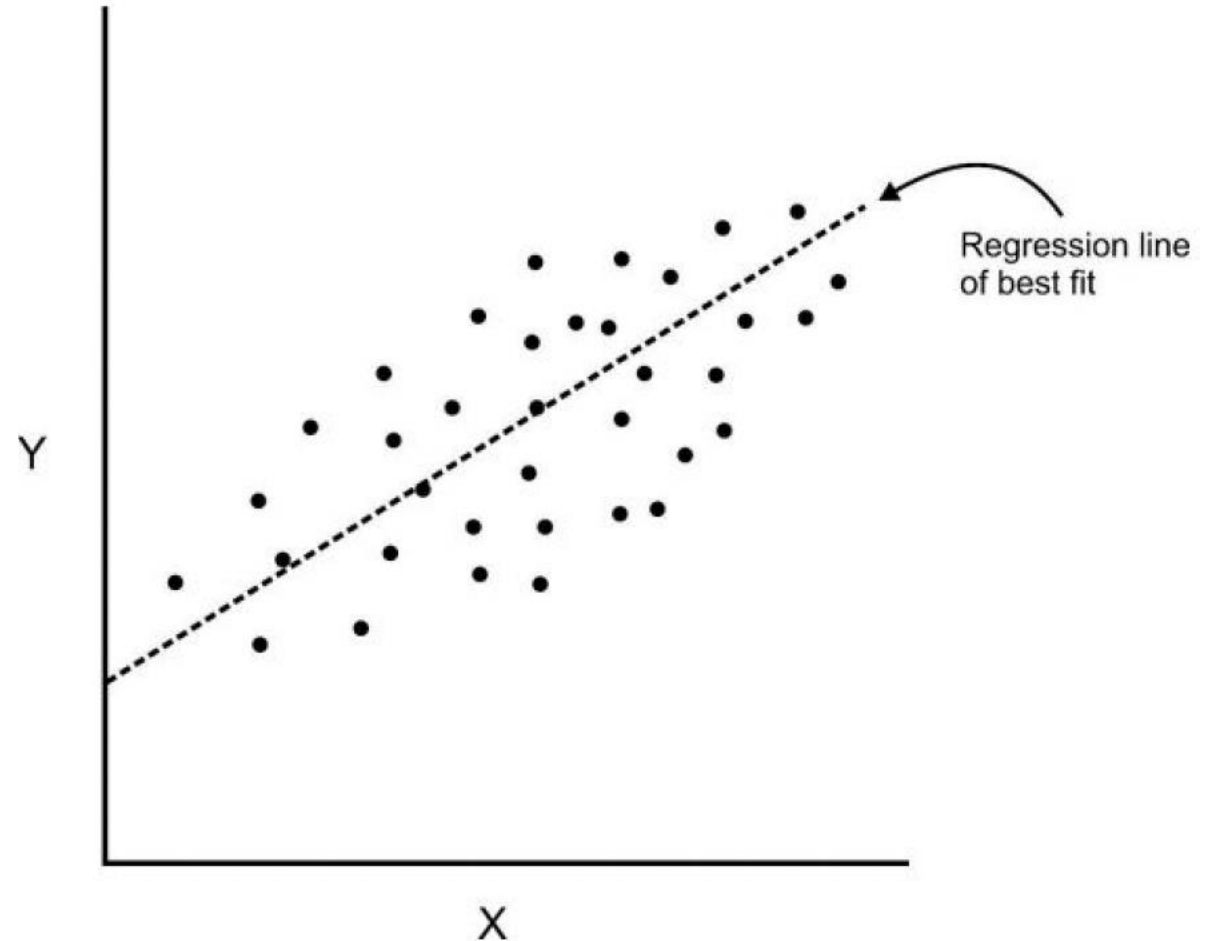
N-Dimensional space

- 28 rows x 28 cols of data = 784 individual cells = 784 dimensions!

[illegible]

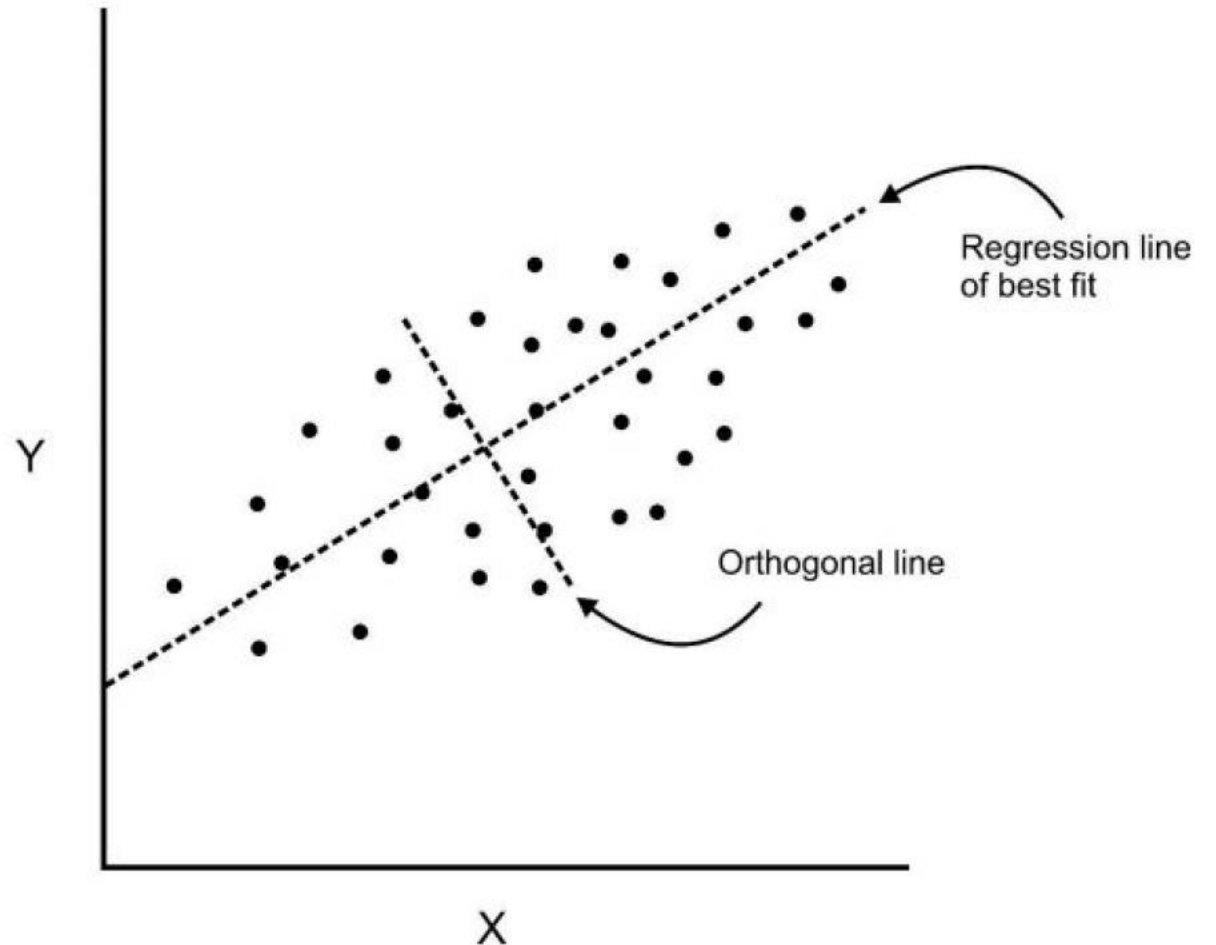
Background to Components

- Here we have some data plotted along two features, x and y .



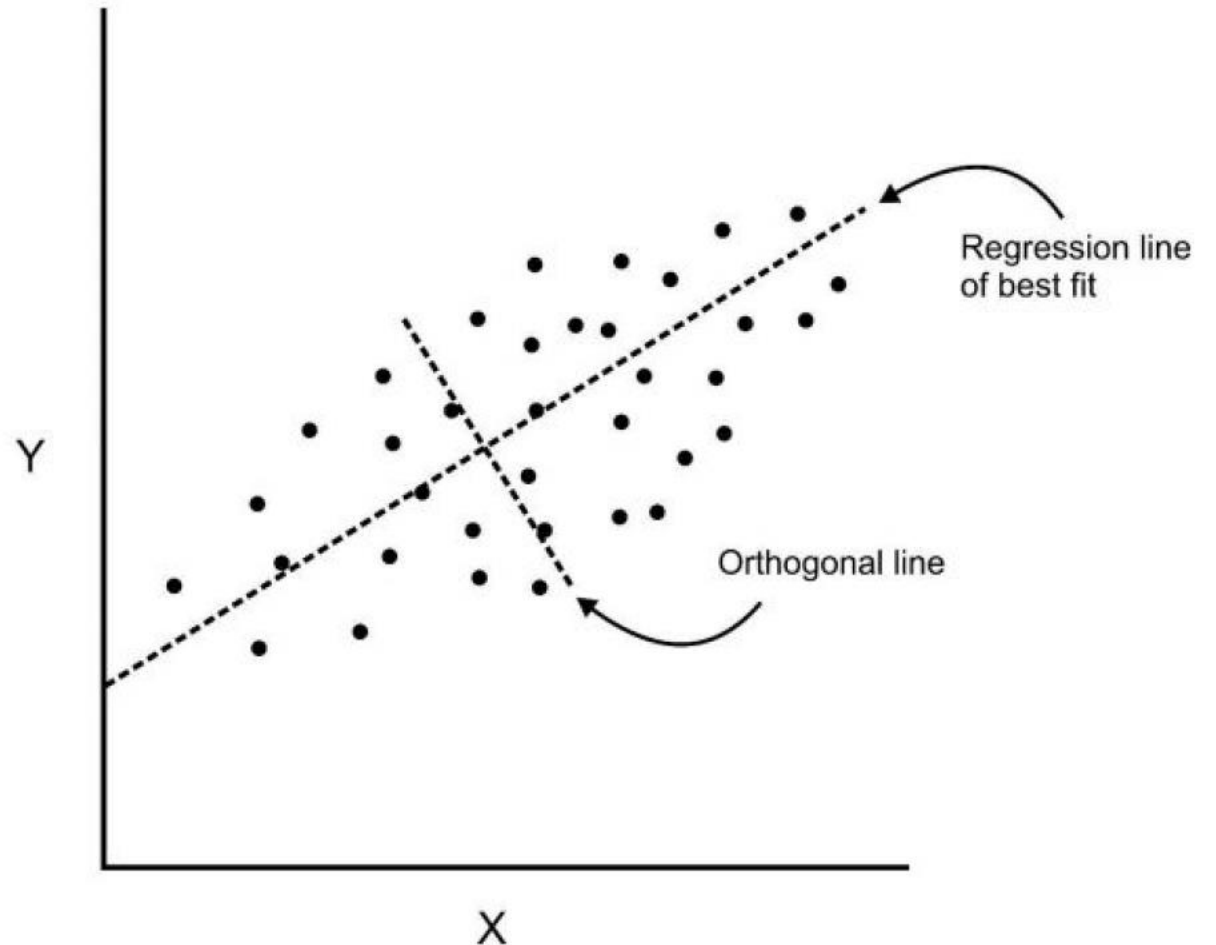
Background to Components

- We can add an orthogonal line.
- Now, we can begin to understand the components!



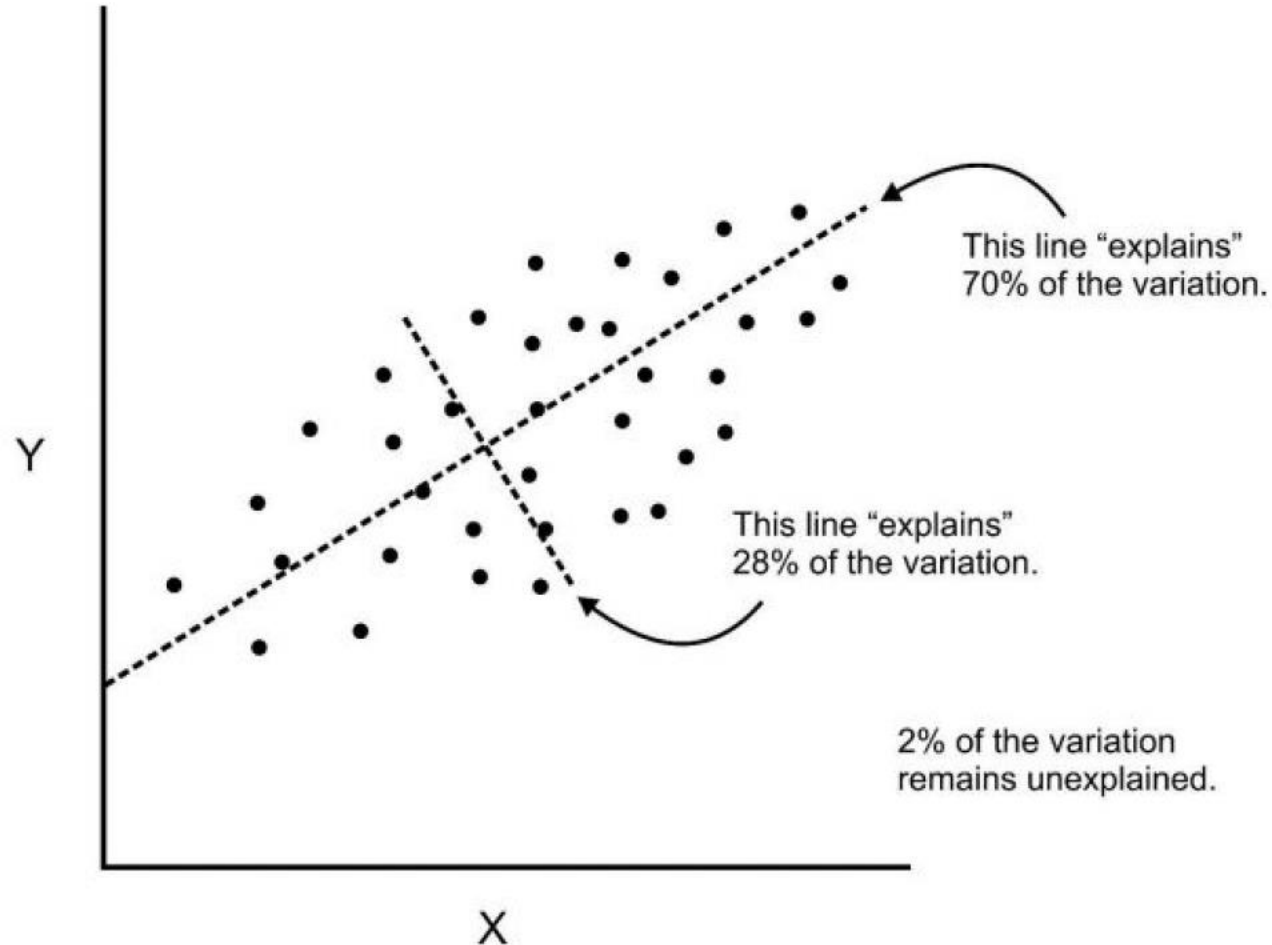
Background to Components

- Components are a linear transformation that chooses a variable system for the dataset such that the greatest variance of the dataset comes to lie on the first axis.



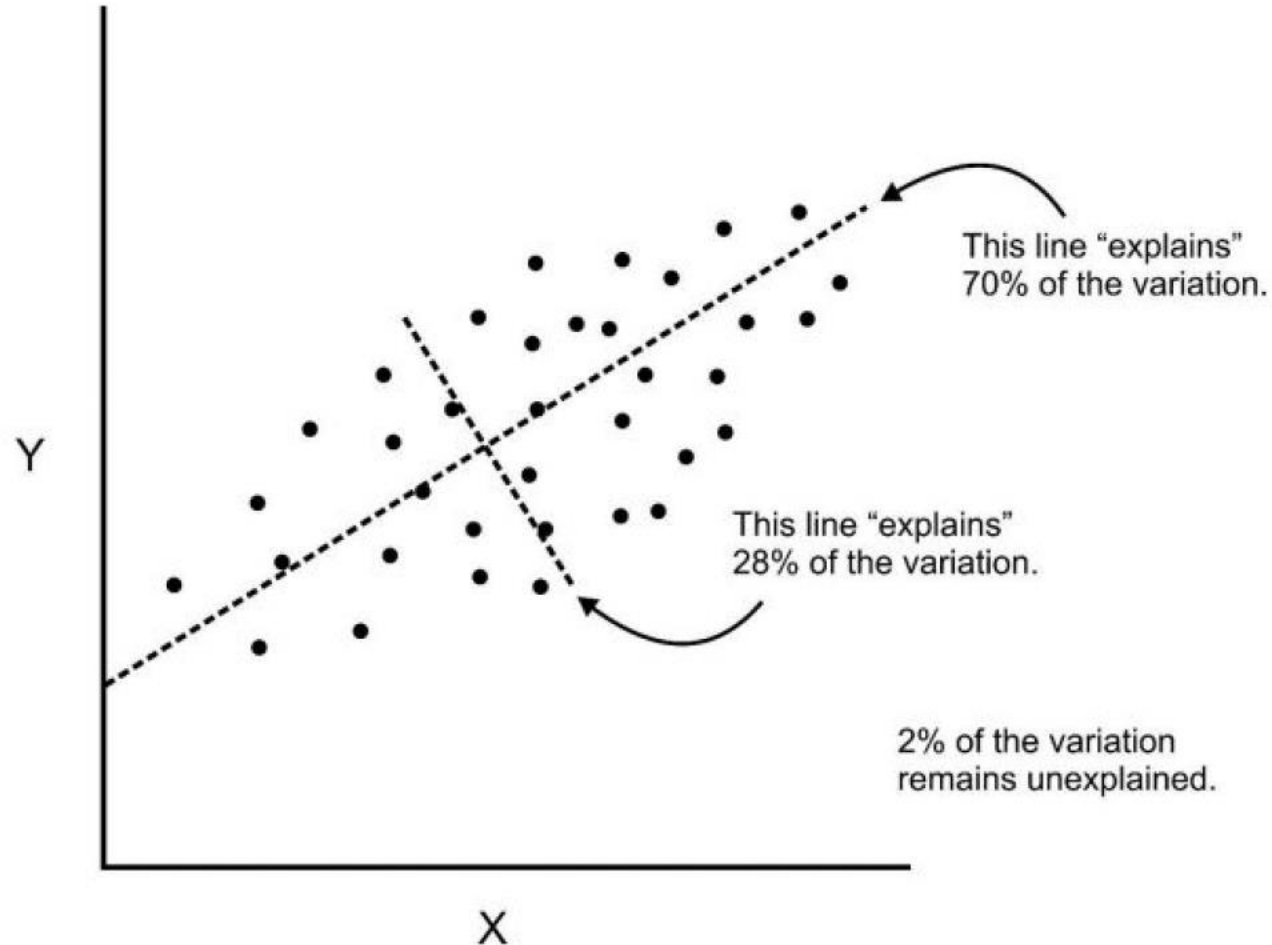
Background to Components

- The second greatest variance on the second axis, and so on...
- This process allows us to reduce the number of variables used in an analysis.



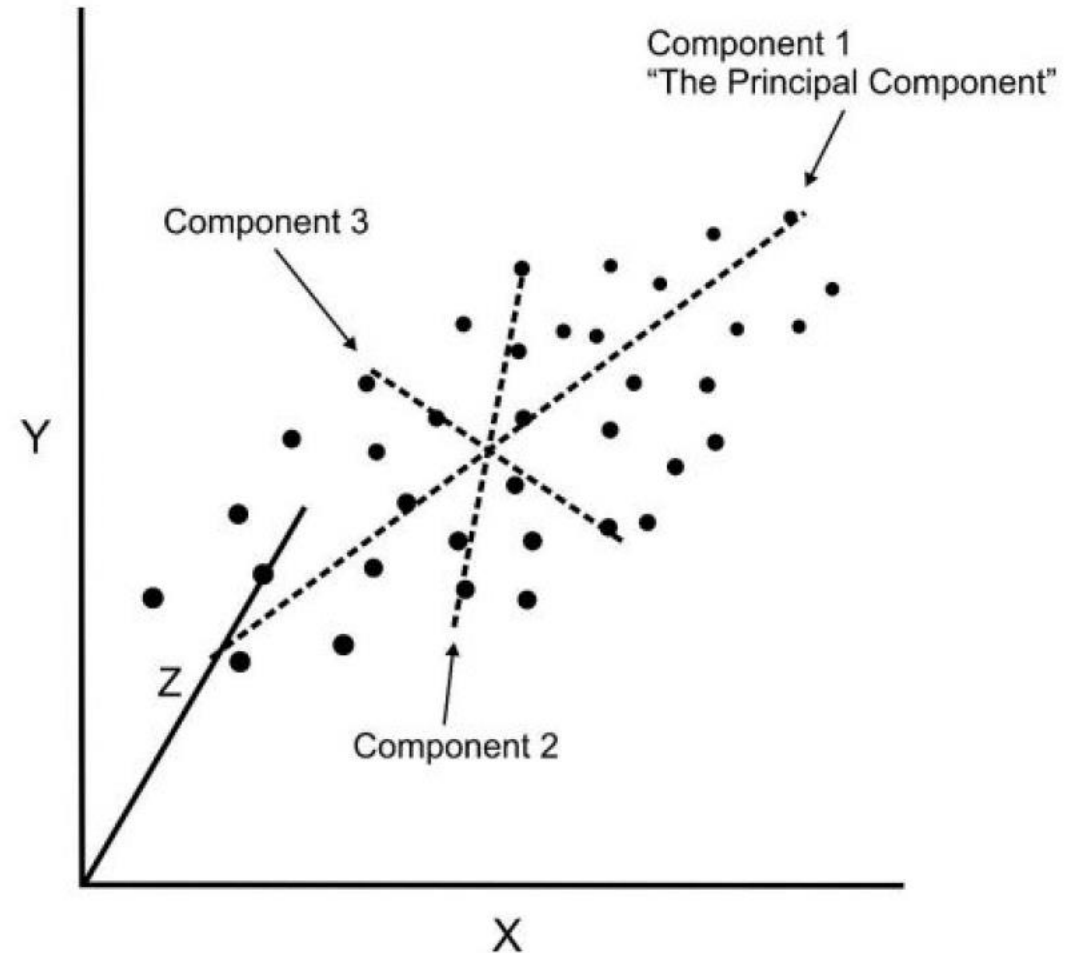
Background to Components

- Note that components are uncorrelated, since in the sample space they are orthogonal to each other.



Background to Components

- We can continue this analysis into higher dimensions



Central aim of PCA

- If we use this technique on a dataset with a large number of variables, we can compress the amount of explained variation to just a few components.
- We also call this **dimensionality reduction**.
- The **most challenging part** of PCA is **interpreting** the components.

Assumptions regarding PCA

- Principal Component Analysis (PCA) relies on several key assumptions, including **linearity, normality, and lack of multicollinearity** in the data. The dataset should also be sufficiently large and **free of significant missing values** for PCA to be effective.
- PCA also requires the variables to be standardised, **as it is sensitive to the scale of the original features**. Fulfilling these prerequisites ensures the principal components extracted are meaningful and the dimensionality reduction is optimal.
- It's also worth noting that PCA is not a deployable model. It is an algorithm used for the purposes of identifying the most explanatory components, so that the dimensionality can then be reduced.

Steps involved in PCA

1. **Standardise** the data (use same scale)
2. Calculate the **covariance matrix**
3. Find the **eigenvalues** and **eigenvectors** (of Covariance Matrix)
4. Choose the **principal components (PC)** to retain based on the explained-variance ratio (aim to retain 95% of variance)
5. **Project** (transform) the high-dimensionality data into a lower-dimensionality space defined by the PC.

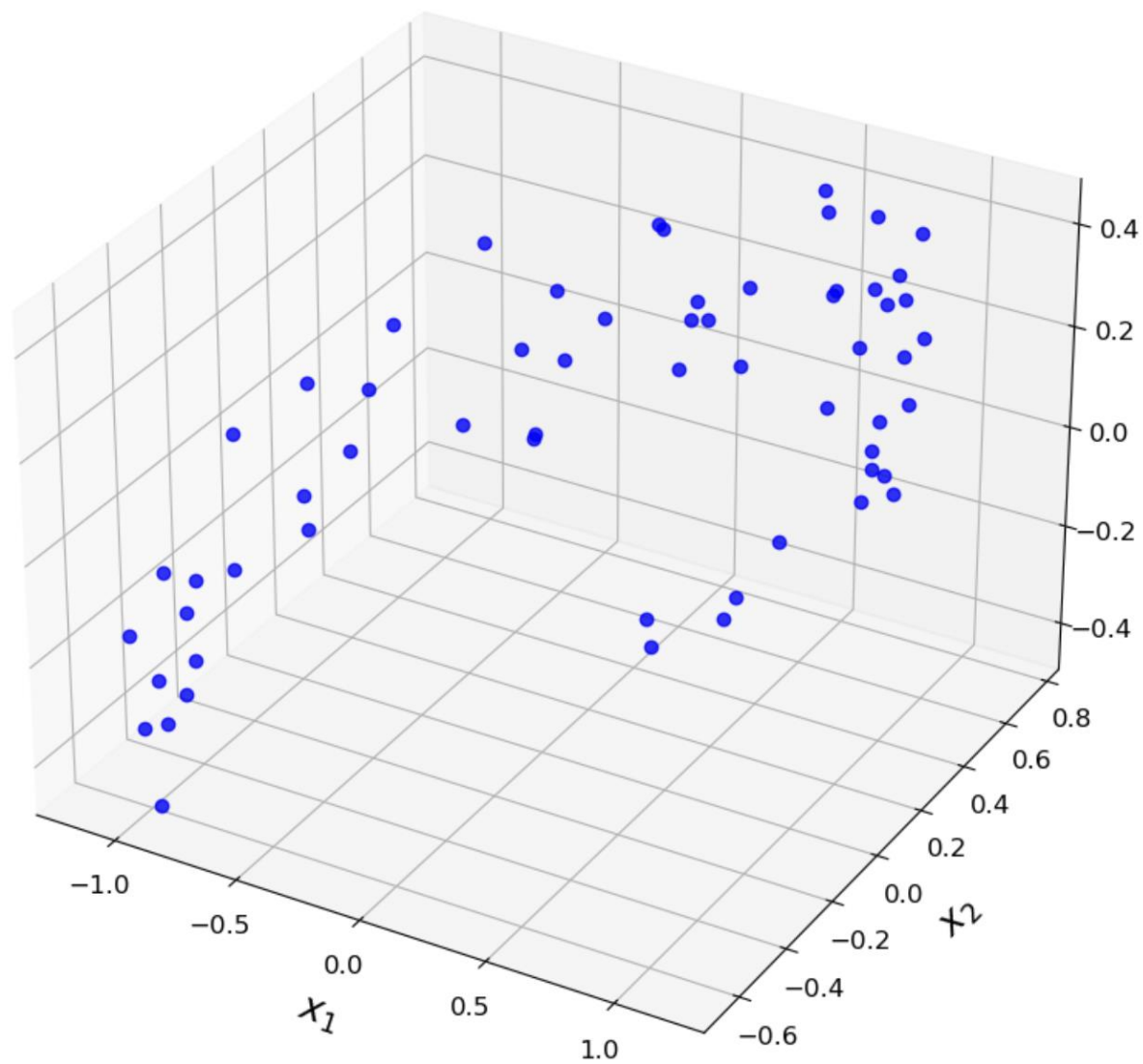
Interpreting principal components

- **Eigenvalues** – indicate **the amount of variance** in the data explained by a component. Larger Eigenvalues correspond to more important principal components (PC).
- **Eigenvectors** – represent the **direction** of the PC. They should show how the original variables are weighted to create each PC.
- **Projection** of the data points onto the PC axes, reveals patterns, clusters, and outliers in the data that are not obvious in high dimensional space.
- **Loadings** – coefficients of the eigenvectors – relative importance to each original variable defining the PC.

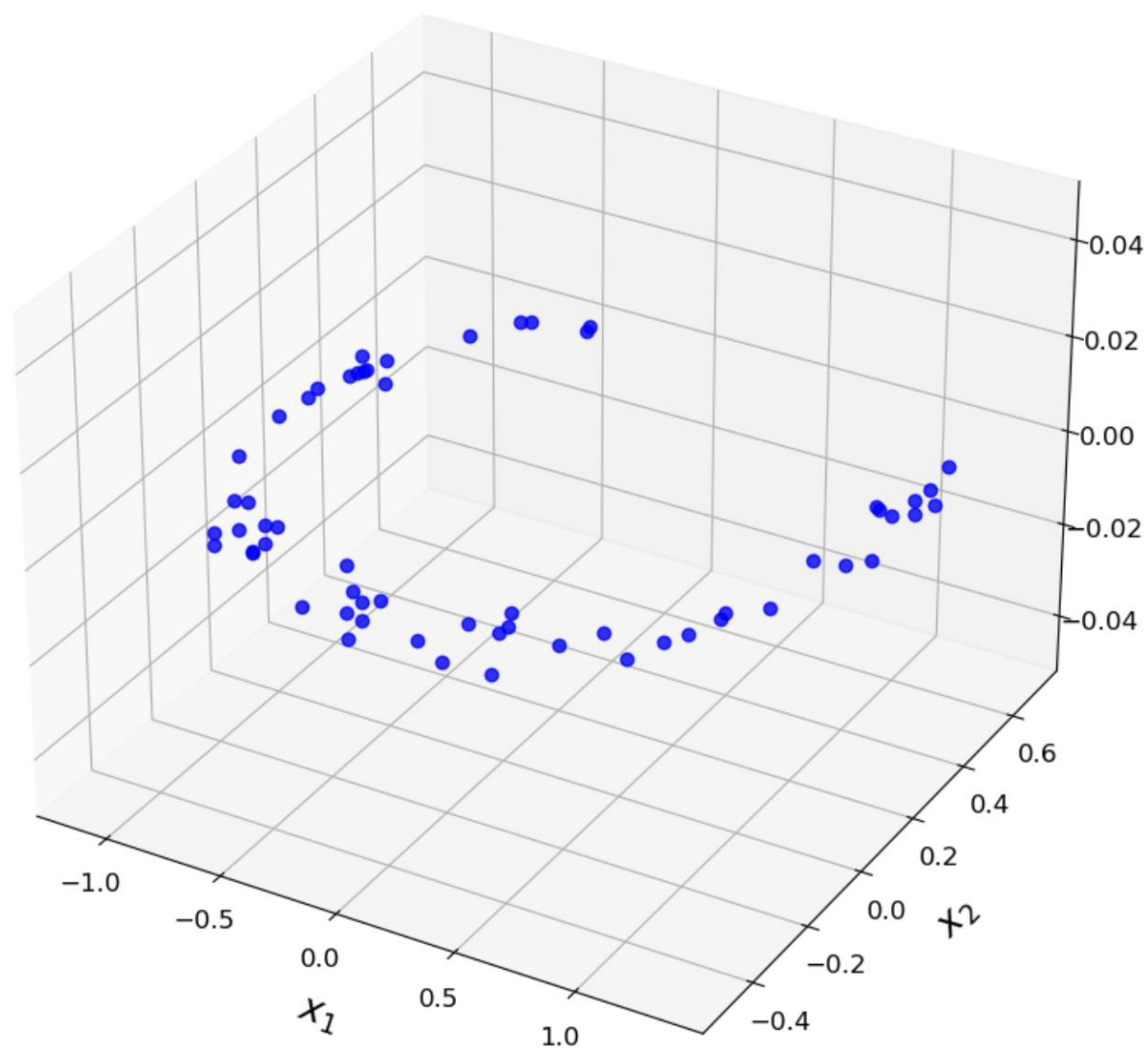
Choosing a number of principal components

- Determining the optimal number of principal components is a crucial step in PCA. This decision balances the need to capture the maximum amount of variance in the data while keeping the model simple and interpretable.
- **2 components** = The minimum number of principal components, which retains the most important information
- **95% variance** = The common threshold for capturing the desired amount of variance in the data
- Scree plots and cumulative explained variance are commonly used to visualise and choose the number of principal components. The goal is to select the fewest components that account for the desired percentage of the total variance, typically around 95%.

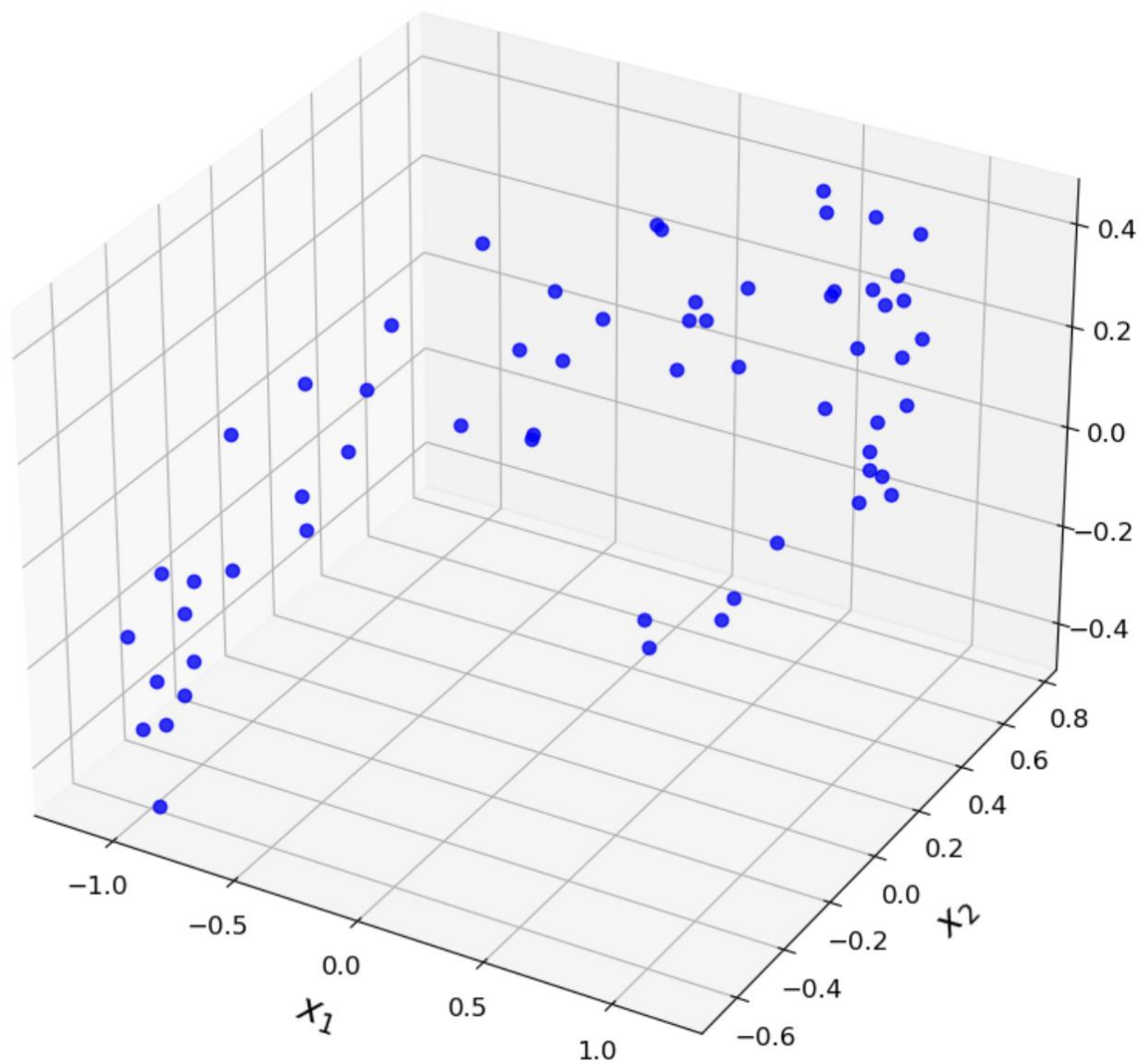
3D Dataset



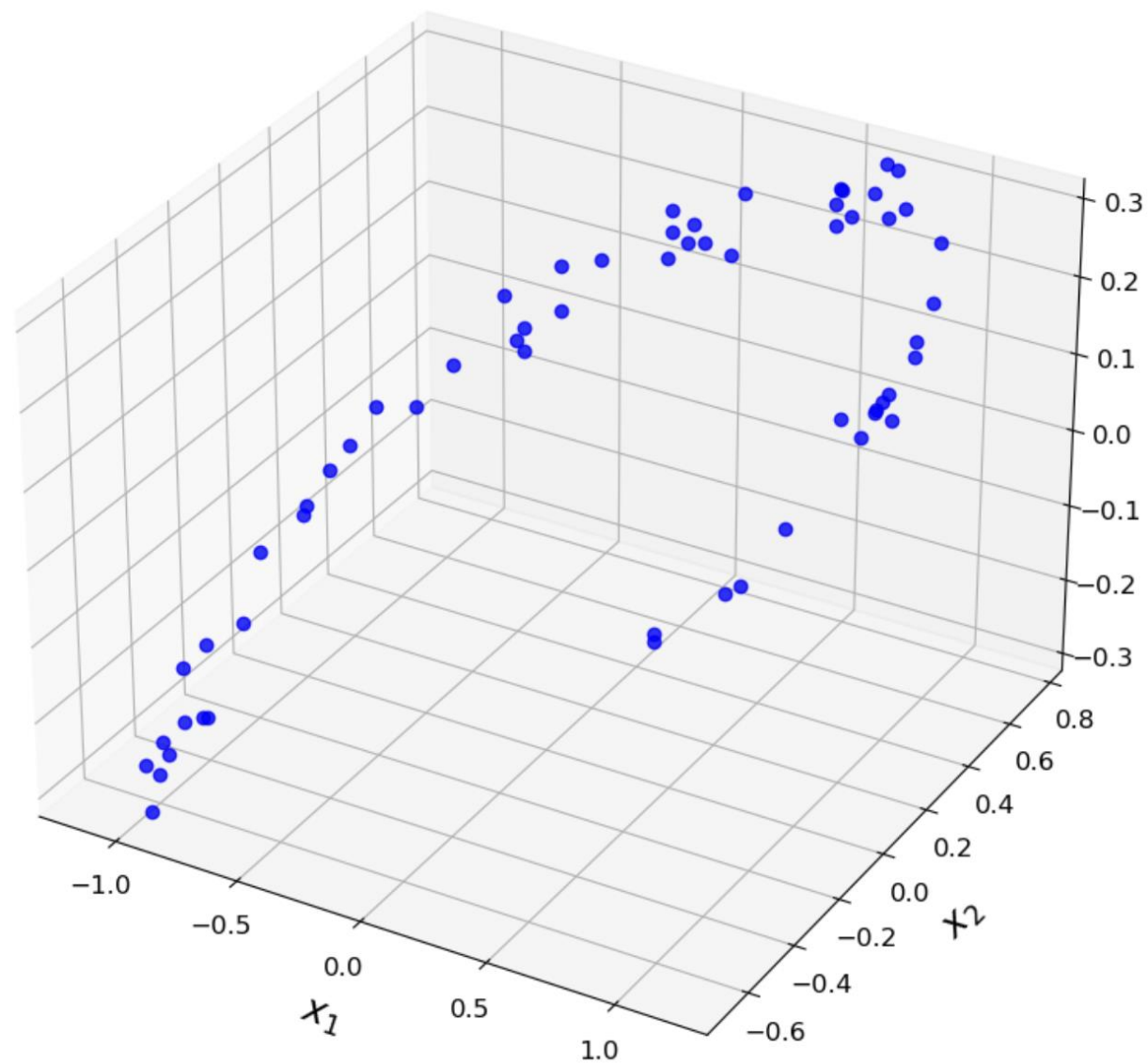
2D Dataset drawn in 3D space



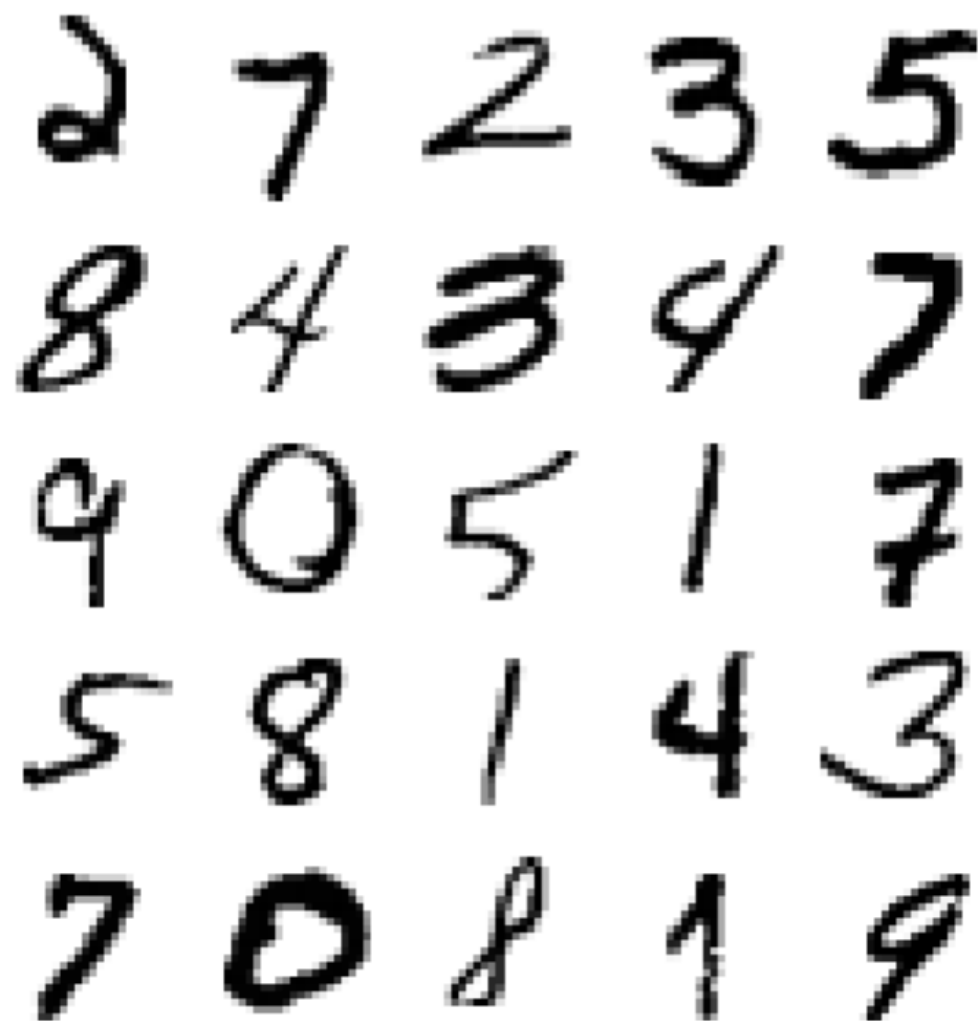
Original 3D Dataset



2D back to 3D Dataset (Inverted)

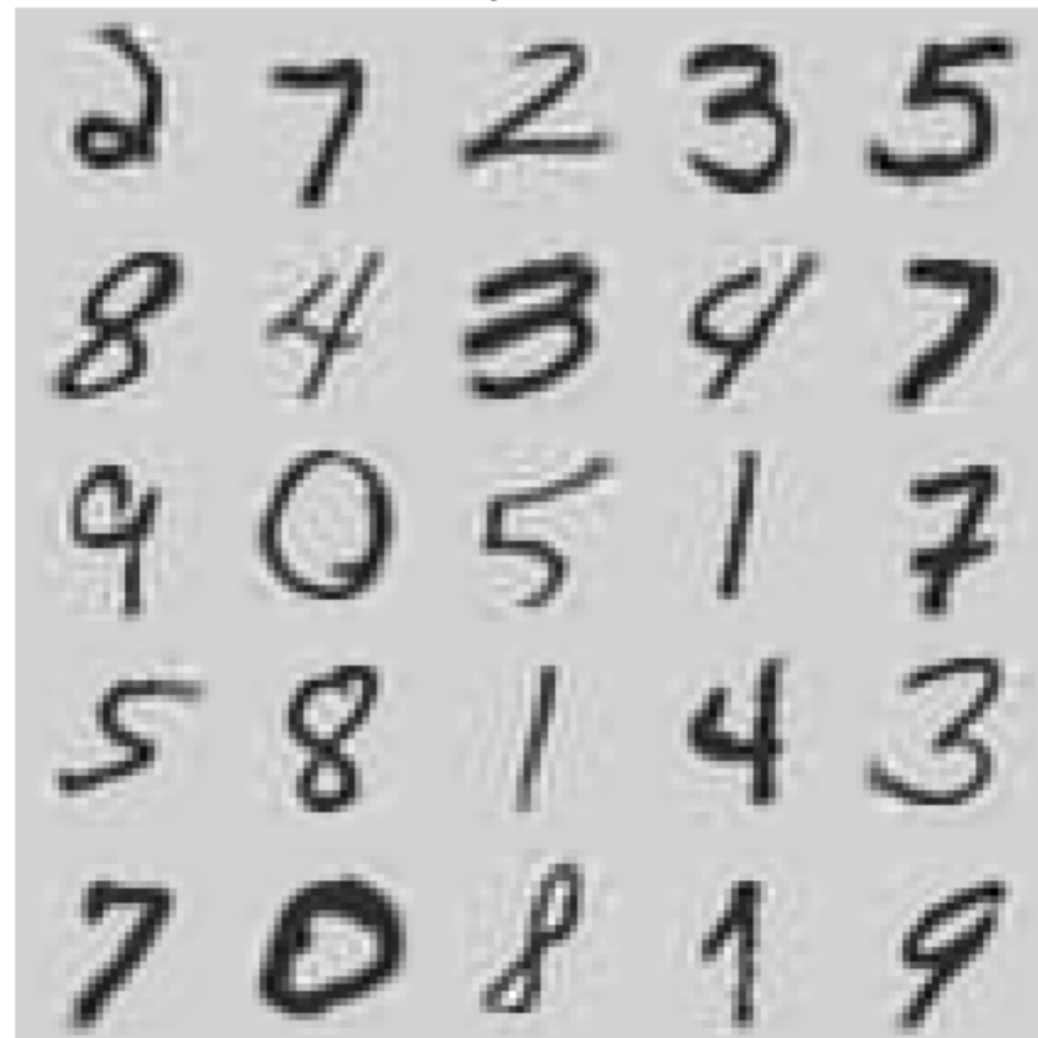


Original



784 dimensions each

Compressed



~150 dimensions each

Implementation with Sklearn

- from **sklearn.decomposition** import **PCA**
- `pca = PCA(n_components=2)`
OR
`pca = PCA(n_components=0.95)`
- `X_reduced = pca.fit_transform(X_train)`
- `print(pca.n_components_)`
- `print(np.sum(pca.explained_variance_ratio))`
- `pca.inverse_transform(X_reduced)`

Implementation with Sklearn

```
pcaTest = PCA(n_components=0.95)  
X_reducedTest = pcaTest.fit_transform(X_test)
```

```
X_reducedTest.shape
```

```
(17500, 153)
```

```
pcaTest.n_components_
```

```
153
```

```
np.sum(pcaTest.explained_variance_ratio_)
```

```
0.9500923711219801
```



Summary of PCA

- **Dimensionality reduction:** PCA can effectively reduce the number of variables in a dataset, making it easier to work with and analyse.
- **Noise removal:** PCA can separate 'signal' from 'noise', extracting the most important features. Reduced storage (compression). Improves model performance (reduces overfitting).
- **Visualisation :** PCA can visualise high-dimensional data in a 2D or 3D space, providing insights into data structure and patterns.
- **Limitations :** PCA assumes linear relationships and therefore, **may not capture non-linear patterns** in the data. It's also sensitive to the **scale** of the input variables.

Dimensionality Reduction

- **PCA** can only build **linear** relationships
- **T-sne** specialise in **non-linear** relationships

